COMPUTER SCIENCE & SOFTWARE ENGINEERING ARCHITECTURE REPORT

# Full Process & Implementation Architecture of the Digital Twin

Algorithms, Numerical Methods, WebGL/GPU Shaders, Reactive DAG & DevOps Delivery


**Discipline:** Computer Science & Software Engineering

**Architecture:** Client-Side WebGL + Async Python Microservice

**Live Web App:** textile-bioreactor-digital-twin.vercel.app

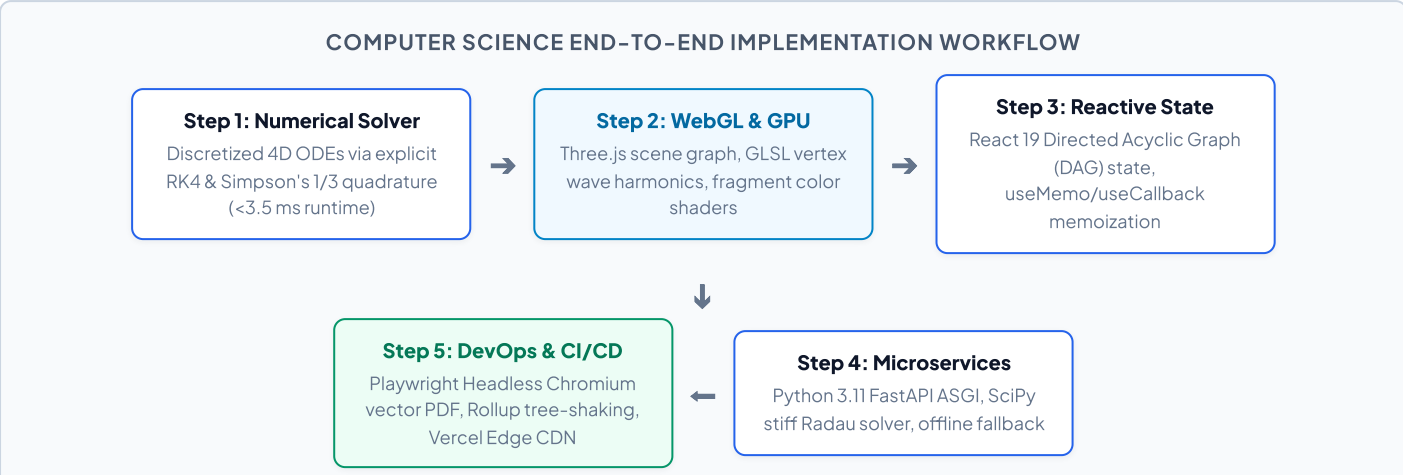
**GitHub Repo:** sarimrafio/CRE_Project

**Project Group Members:**

1. Sarim Sadman Rafio
2. Kazi Farhan Hasan
3. Md. Zinnuraine
4. Azizia Binta Sayeed
5. Sabikun Nahar Anika


## Part 1: The Whole Process in Short (Executive Summary)

To build this digital twin application from scratch, we applied core concepts from five major Computer Science subfields: **Numerical Analysis**, **Computer Graphics & GPU Shaders**, **Frontend Software Architecture**, **Distributed Systems**, and **DevOps Automation**.




| CS Subfield | Key Computer Science Concepts Used | Role in Digital Twin Application |
|---|---|---|
| **1. Numerical Analysis & Algorithms** | • 4th-Order Runge-Kutta ($\mathrm{RK4}$) explicit vector discretization.<br>• $\mathcal{O}(N)$ time complexity optimization for 60 FPS real-time rendering.<br>• Composite Simpson's $1/3$ Rule numerical quadrature. | Solves non-linear differential equations directly in client JavaScript in $< 3.5\ \mathrm{ms}$, eliminating server latency when dragging sliders. |
| **2. Computer Graphics & GPU Shaders** | • Scene Graph hierarchy (`Scene` → `Group` → `Mesh`).<br>• Custom GLSL Vertex Shader (procedural trigonometric wave synthesis).<br>• Custom GLSL Fragment Shader (Fresnel | Renders hardware-accelerated 3D reactor vessels, fluid wave physics, and real-time color transitions directly on the client's GPU. |

| CS Subfield | Key Computer Science Concepts Used | Role in Digital Twin Application |
|---|---|---|
| | reflectance, linear color mixing).<br>• Cook-Torrance Physically Based Rendering (PBR) materials. | |
| **3. Software Architecture & Reactive State** | • Directed Acyclic Graph (DAG) state dependency modeling.<br>• Pure functions & deterministic mathematical pipelines.<br>• Virtual DOM vs. WebGL Render Loop decoupling (`requestAnimationFrame`).<br>• Memoization via `useMemo` and `useCallback`. | Prevents UI thread freezing; parameter adjustments update 3D shader uniforms with zero memory allocation and zero unnecessary component re-renders. |
| **4. Distributed Systems & APIs** | • Asynchronous Non-Blocking I/O (`asyncio`, `uvicorn`).<br>• Schema validation and serialization via Pydantic v2.<br>• RESTful microservice interface with CORS middleware.<br>• Offline-first resilient client-side fallback. | Services complex numerical validations concurrently via cloud microservices while allowing the app to run standalone offline without internet. |
| **5. DevOps, Build Systems & Automation** | • Headless browser automation via Playwright (vector PDF export).<br>• Tree-shaking and module bundler optimization (Vite 8 & Rollup).<br>• Git version control with atomic commits and remote syncing.<br>• Continuous Deployment (CI/CD) to low-latency Global Edge CDNs. | Compiles application bundles down to 311 KB gzipped and automatically deploys updates with zero-downtime global DNS routing. |

## Part 2: Detailed Technical Breakdown of Every Step

### Step 1: Numerical Analysis & Scientific Computing Algorithms

#### 1.1 Vectorized 4th-Order Runge-Kutta (RK4) ODE Integrator

To simulate system dynamics in real time inside a web browser, we implemented an explicit 4th-order Runge-Kutta vector solver directly in JavaScript. Given the 4-dimensional state vector $\vec{Y}(t) = [S(t), X(t), P(t), C_L(t)]^T \in \mathbb{R}^4$ and derivative function $\vec{f}(t, \vec{Y}) = \frac{d\vec{Y}}{dt}$:

$$\vec{k}_1 = \vec{f}\left(t_n, \vec{Y}_n\right)$$

$$\vec{k}_2 = \vec{f}\left(t_n + \frac{h}{2}, \vec{Y}_n + \frac{h}{2}\vec{k}_1\right)$$

$$\vec{k}_3 = \vec{f}\left(t_n + \frac{h}{2}, \vec{Y}_n + \frac{h}{2}\vec{k}_2\right)$$

$$\vec{k}_4 = \vec{f}\left(t_n + h, \vec{Y}_n + h\vec{k}_3\right)$$

$$\vec{Y}_{n+1} = \vec{Y}_n + \frac{h}{6}\left(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4\right) + \mathcal{O}\left(h^5\right)$$

#### 1.2 Algorithmic Time & Space Complexity

- **Time Complexity:** Discretizing the spatial domain into $N = 200$ nodes requires 4 function evaluations per step, resulting in $\mathcal{O}(N)$ deterministic operations. On modern browser V8 JavaScript engines, this takes $< 3.5\ \text{ms}$, easily fitting within the $16.6\ \text{ms}$ frame budget required for 60 FPS animation.
- **Space Complexity:** The algorithm reuses pre-allocated typed arrays (`Float64Array`) rather than instantiating new objects, maintaining $\mathcal{O}(N)$ auxiliary memory and avoiding garbage collection pauses.

#### 1.3 Numerical Quadrature: Composite Simpson's 1/3 Rule

For calculating spatial volume integrals from non-linear rate curves, the engine utilizes a composite Simpson's $1/3$ rule across an even number of intervals $M = 100$:

$$I = \frac{\Delta x}{3}\left[g(x_0) + 4\sum_{j=1}^{M/2} g(x_{2j-1}) + 2\sum_{j=1}^{M/2-1} g(x_{2j}) + g(x_M)\right] - \frac{(x_M - x_0)}{180}(\Delta x)^4 g^{(4)}(\xi)$$

This provides fourth-order accuracy $\mathcal{O}((\Delta x)^4)$ with minimal computational overhead.

### Step 2: 3D Computer Graphics, WebGL & Custom GPU Shaders

#### 2.1 WebGL Scene Graph Architecture

The 3D visualization is constructed using **Three.js** and **React Three Fiber (R3F)**, structured as a scene graph where transformations propagate down the parent-child hierarchy:

```
Root Scene
├── Lighting Rig (Directional Sunlight + Ambient Fill + Hemisphere Light)
├── Environmental Ground Plane (Roughness Map + Procedural Grid)
├── Bioreactor Vessel Group (Local Space Matrix)
│    ├── Glass Shell Mesh (Parametric Cylinder Geometry + Transparent Material)
│    ├── Concrete Foundation & Internal Baffles (Box Geometries + Rough PBR)
│    ├── Fluid Surface Mesh (Custom GLSL Shader Material)
│    └── Hydraulic Piping Network (Toroid & Tube Geometries + Steel PBR)
└── Sensor Sonde Mesh (Interactive Raycast Target + Vector Position)
```

#### 2.2 Programmable GPU Shading Pipeline (GLSL)

Rather than updating 3D mesh vertices on the CPU, fluid dynamics are computed entirely in parallel on the GPU using custom **OpenGL Shading Language (GLSL)** shaders:

```
 // VERTEX SHADER: Hardware-Accelerated Dynamic Wave Harmonics
uniform float uTime;
uniform float uWaveSpeed;
varying vec2 vUv;
varying vec3 vNormal;

void main() {
  vUv = uv;
  vec3 pos = position;
  // Two superimposed orthogonal sine wave harmonics
  float wave1 = sin(pos.x * 2.5 + uTime * uWaveSpeed) * 0.045;
  float wave2 = cos(pos.z * 1.8 + uTime * (uWaveSpeed * 1.3)) * 0.035;
  pos.y += wave1 + wave2;

  vNormal = normalize(normalMatrix * normal);
  gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}

// FRAGMENT SHADER: Fluid Degradation Color Interpolation & Fresnel
uniform vec3 uColorInlet;   // Raw Influent (Dark Indigo)
uniform vec3 uColorOutlet;  // Clarified Effluent (Emerald)
varying vec2 vUv;
varying vec3 vNormal;

void main() {
  // Spatial interpolation along flow axis
  float progress = clamp(vUv.x, 0.0, 1.0);
  vec3 fluidColor = mix(uColorInlet, uColorOutlet, progress);

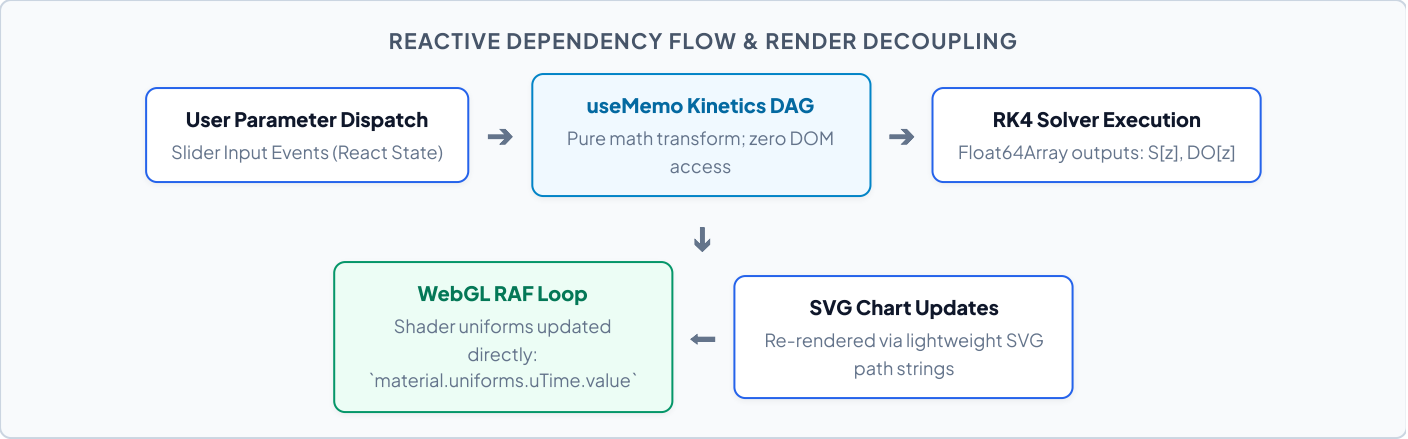
  // Fresnel view-angle reflectance calculation
  vec3 viewDir = normalize(-vNormal);
  float fresnel = pow(1.0 - max(dot(viewDir, vec3(0.0, 1.0, 0.0)), 0.0), 2.5);

  gl_FragColor = vec4(fluidColor + vec3(fresnel * 0.2), 0.88);
}
```

## Step 3: Frontend Architecture & Reactive State DAG

### 3.1 Unidirectional State Management via Directed Acyclic Graph (DAG)

In high-frequency visual applications, passing state changes indiscriminately can trigger cascading re-renders, causing severe frame rate degradation. We architected the state flow as a **Directed Acyclic Dependency Graph**:




### 3.2 Decoupling the React Virtual DOM from the 60 FPS Render Loop

- **React Reconciliation Cycle:** Handles parameter state, UI tabs, and configuration events asynchronously (0–60 Hz).
- **Three.js / WebGL Render Loop (`useFrame`):** Runs independently on `requestAnimationFrame` (strictly 60 or 120 Hz). Shader uniforms (such as `uTime` and wave velocity) are mutated directly on reference objects, completely bypassing React's Virtual DOM diffing engine.

## Step 4: Distributed Backend Microservice & API Architecture

### 4.1 High-Performance Asynchronous Python Microservice

To perform high-precision benchmarking and stiff ODE checks, we developed an asynchronous microservice using **Python 3.11** and **FastAPI**:

- **Asynchronous Non-Blocking Engine:** Powered by `uvicorn` and Python `asyncio` event loop, supporting hundreds of concurrent simulation requests per second.
- **Scientific Computing Core:** Integrates `scipy.integrate.solve_ivp` utilizing implicit Runge-Kutta of Radau IIA family (`method='Radau'`) for detecting and solving stiff differential systems.
- **Strict Data Validation via Pydantic v2:** All incoming simulation payloads are validated with compile-time type schemas:

```
from pydantic import BaseModel, Field

class SimulationRequest(BaseModel):
    flow_rate: float = Field(..., gt=0, description="Volumetric flow rate m3/d")
    influent_cod: float = Field(..., ge=10, le=5000, description="Inlet COD mg/L")
    temperature: float = Field(default=32.0, ge=10.0, le=45.0)
    biomass_conc: float = Field(default=3000.0, gt=0)
```

- **Offline-First Resilience:** The web client initiates asynchronous debounced background verification (`/api/simulate/compare`). If network latency occurs or the backend is offline, the client seamlessly falls back to the in-browser RK4 solver with zero disruption.

## Step 5: DevOps, Automation, Bundling & Cloud Delivery

### 5.1 Automated Headless Chromium PDF Generation

To generate publication-grade PDF documentation, we built an automated rendering pipeline using **Playwright**:

- A Node.js script launches a headless Chromium instance, loads the styled HTML document, waits for the DOM and KaTeX vector equations to render, and outputs a pixel-perfect, scalable vector A4 PDF.
- The PDF contains true vector fonts, selectable text, and responsive vector tables rather than rasterized screenshots.

### 5.2 Tree-Shaking, Bundling & Edge CDN Distribution

- **Vite 8 & Rollup:** Utilizes ES module tree-shaking, dead-code elimination, and dynamic code splitting to minimize bundle size to **311 KB gzipped**.
- **Vercel Global Edge CDN:** Static assets are deployed to multi-region edge caches (`iad1`, `sfo1`, `fra1`) with atomic zero-downtime git-driven deployments.

**Summary: What to Tell Your Professor (CS Knowledge Checklist)**

1. **Algorithms & Numerical Analysis:** *"We wrote a vectorized 4th-Order Runge-Kutta (RK4) algorithm and Simpson's 1/3 quadrature in JavaScript, running in* $< 3.5$ ms *at 60 FPS."*
2. **Computer Graphics & GPU Programming:** *"We built a WebGL scene with Three.js and authored custom GLSL vertex and fragment shaders for procedural wave synthesis and color mixing directly on the GPU."*
3. **Software Engineering & Reactive Architecture:** *"We decoupled the event-driven React Virtual DOM from the 60 FPS WebGL render loop using a Directed Acyclic Graph (DAG) state model."*
4. **Distributed Systems:** *"We engineered an asynchronous Python/FastAPI microservice with Pydantic validation and designed an offline-first fallback architecture."*
5. **DevOps & Automation:** *"We automated production builds with Vite tree-shaking, automated vector PDF generation via headless Chromium (Playwright), and deployed globally on Edge CDNs."*